EE 474 Lab Part 3: Open-Loop and Closed-Loop Control (Velocity Servo)

Contents

1 Introduction **2**
1.1 Plant modeling and linearization . . . 2
1.1.1 A note on capturing the plant model . . . 2
1.1.2 Linearization of a nonlinear plant . . . 2
1.2 Goals of lab 3 . . . 5
1.3 Introduction to open- and closed-loop control . . . 5
1.3.1 Open-loop control . . . 5
1.3.2 Closed-loop control . . . 6
1.4 Determining the DC gain, K_{uv}, and designing open-loop control . . . 6
1.5 Open-loop controller design . . . 7
1.6 Designing P-type closed-loop control . . . 8
1.7 Designing P-type closed-loop control with feed forward . . . 8
1.8 Prelab questions . . . 9

2 In Lab **11**
2.1 Basic setup and sensors . . . 11
2.1.1 Cabling the SRV02 . . . 11
2.1.2 Setting up sensors . . . 11
2.2 System identification, determine the steady-state model of the SRV02 . . . 11
2.3 Open-Loop Control . . . 12
2.4 Closed-loop, P-type control . . . 13
2.5 Closed-loop, P-type control with feed forward . . . 14

3 Write-up **15**
3.1 Principal Values and Graphs . . . 15
3.2 Questions . . . 15

4 Appendices **17**
4.1 Example script to set parameters . . . 17
4.2 Fitting parameters to data . . . 18

1 Introduction

1.1 Plant modeling and linearization

Modeling is an important part of control system analysis and realization. Obtaining accurate model parameters is a substantial part of the cost of practical control systems. In this section, some broad modeling concepts are introduced that will be important for working with the SRV02 throughout the semester.

1.1.1 A note on capturing the plant model

It may seem as if the dynamic model of a system, such as $G_{uv}(s)$ for the SRV02, should be fixed, capturing all available knowledge of the dynamics of the system. But in practice it is often the case that model complexity varies, depending on the extent of available knowledge and budget for determining accurate parameters. In an oil refinery with 10,000 control points - valves, pumps, heating units - the budget may restrict modeling for each control point to simple models, perhaps as simple as Eqn (1), below. In contrast, in large aerospace programs, such as the Boeing 787 or F-35, where the system is complex and many identical units will be built, and where a small difference in performance can bring significant value, tens of millions of dollars and hundreds of person-years of effort may be spent developing a detailed dynamic model.

During the EE 474 lab sequence, you will be working extensively with the model of the SRV02, starting with a very simple model and progressively developing more detail. In this lab, you will work with the simplest model,

$$G_{uv}(s) = K_{uv} \qquad \left[\frac{\text{radians/second}}{\text{volt}}\right] \tag{1}$$

where $G_{uv}(s)$ is the transfer function of the motor servo, and K_{uv} [radians/sec/volt] is a constant that relates the applied voltage to the *steady-state* rotation rate.

1.1.2 Linearization of a nonlinear plant

Plant is a name often given to the process or system being controlled. A single-input, single-output plant is illustrated in figure 1.

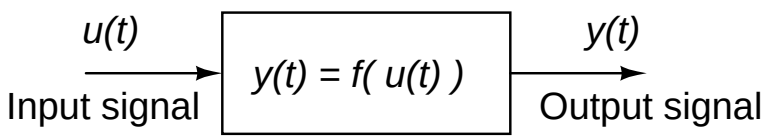


Figure 1: Plant with input and output signals.

The plant of figure 1 is shown with governing equation

$$y(t) = f(u(t)) \,. \tag{2}$$

For a ***linear*** system, the governing equation can be simplified to a linear equation, such as

$$y(t) = K_{uv}\, u(t) \tag{3}$$

where K_{uv} is a constant gain relating the input to the output.

However, many systems, including the SRV02 motor servo we use in lab, are governed at least partially by nonlinear effects. In the SRV02, the main nonlinearity is friction.

A possible non-linear input-output relationship is seen in figure 2. To work with nonlinear system elements and apply linear system theory, we ***linearize*** the response curve of the system element about an ***operating point***.

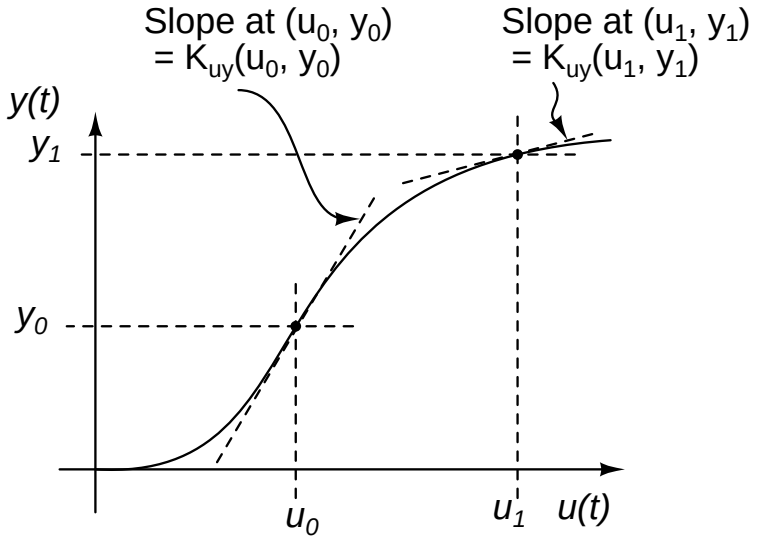


Figure 2: Possible non-linear input-output relationship of a system element (this curve might be typical of a valve, for example an engine throttle). Two operating points are shown.

Linearization about an operating point involves two parts:

1. The operating point, which is the level on the input-output curve about which the system will be linearized. The operating point can be expressed as an input-output pair. Points (u_0, y_0) and (u_1, y_1) are two candidate operating points.

2. The slope of the input-output curve at the operating point is the gain of the system element at that operating point. The gain at the operating point can be expressed as $K_{uv}(u_0, y_0)$.

Starting with the nonlinear input-output relationship

$$y(t) = f(u(t)) \tag{4}$$

and an operating point (u_0, y_0) and expressing the input and output signals as deviations about the operating point,

$$u(t) = u_0 + \delta u(t)\,, \qquad y(t) = y_0 + \delta y(t) \tag{5}$$

and with $K_{uv}(u_0, y_0)$ as the slope of the input-output curve at (u_0, y_0), then the deviation in the output is given from the deviation in the input by:

$$\delta y(t) \simeq K_{uv}(u_0, y_0)\, \delta u(t) \tag{6}$$

Equation (6) is a *linear* relationship between the input and output *deviations*. Putting Equations (5) and (6) together, we may write:

$$y(t) \simeq y_0 + K_{uv}(u_0, y_0)\, \delta u(t) \tag{7}$$

Equations (6) and (7) show three important details:

1. The linearization applies to deviations about the operating point,

2. The gain depends on the operating point, and

3. The linearization is approximate.

However, linearization is so widely used in control system analysis and design that a short hand is often used. Starting with the equation in small deviations,

$$\delta y(t) \simeq K_{uv}(u_0, y_0)\, \delta u(t) \qquad \text{(Eqn (6), repeated)}$$

the linearized input-output relationship is written as:

$$y(t) = K_{uv}\, u(t) \tag{8}$$

where the short hand is to omit the details of the operating point, and using the $y(t)$ and $u(t)$ symbols to represent the deviations from the operating point, rather than the absolute magnitude of the signals. As an example consider aircraft flight controls. Variable $h(t)$ [feet] might represent altitude. If the assigned flight level is 30,000 feet and the aircraft is currently 10 feet below that level, in the controller the altitude will be represented as $h(t) = -10$ [feet] , even though the absolute altitude is 29,990 feet (and an absolute altitude of -10 feet could be very problematic!) Routinely, controls engineers work with the deviations from operating point, but leave off the δ′s that would appear throughout the equations; but we must remember that when there has been a linearization, the three important details apply and equations of the form of equations (6) and (7) are the true or absolute-coordinates description of the system.

1.2 Goals of lab 3

The goals of lab 3 are:

1. Determine the linearized model of the SRV02, by measuring the steady-state velocity as a function of applied voltage, to develop data comparable to that of figure 2, setting an operating point, and determining the DC Gain K_{uv} .

2. Design open and closed-loop controllers for the SRV02 and test these with respect to square wave tracking.

1.3 Introduction to open- and closed-loop control

Controllers generate and send a command signal that regulates equipment or a process. Examples are industrial process controls, vehicle control, such as car cruise control, or regulation of robot motions. Controllers can be classified into two groups: open- and closed-loop control.

1.3.1 Open-loop control

Open-loop control is characterized by the absence of a feedback path, that is, the absence of a signaling pathway for the output signal to affect the input. An open-loop control system is illustrated in figure 3. The signals of the system in figure 3 are listed in table 1.

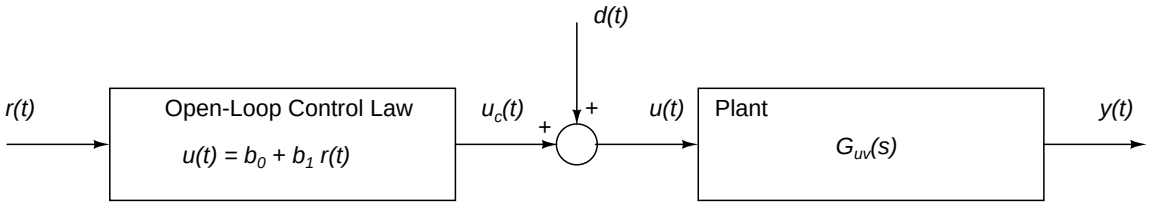


Figure 3: Open-loop velocity control with disturbance input.

$r(t)$	reference input, [radians/second]
$u_c(t)$	motor command (controller output), [volts]
$d(t)$	disturbance input (shown in parallel with the motor command), [volts-equivalent]
$u(t)$	Total effective command to the motor [volts]
$y(t)$	system output, [radians/second]

Table 1: Definitions of signals in figure 3, with units and description corresponding to the SRV02.

1.3.2 Closed-loop control

A closed-loop controller is characterized by the utilization of feedback, that is, a pathway for the output signal to affect the input. A basic single-loop, closed-loop control system is illustrated in figure 4. The additional signals are listed in table 2.

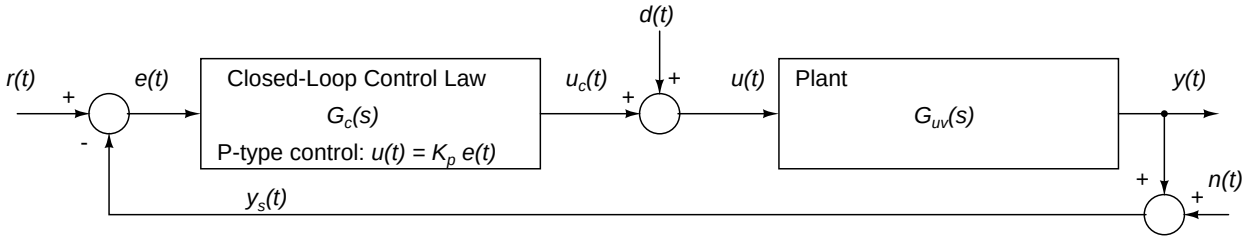


Figure 4: Closed-loop velocity control with velocity sensing and disturbance input.

$e(t)$	the error signal, it is the difference between $r(t)$ and $y(t)$, [radians/second]
$n(t)$	sensor noise signal, [radians/second]
$y_s(t)$	sensed velocity signal, [radians/second]

Table 2: Signals added for the closed-loop system, with units and description corresponding to the SRV02.

In lab 3 you will approximate the plant model $G_{uv}(s)$ with a constant:

$$G_{uv}(s) = K_{uv} \tag{9}$$

where K_{uv} is the DC gain of the motor servo. You will identify (measure) K_{uv} and design open-loop and closed-loop controllers to cause the motor output $y(t)$ to track the reference input $r(t)$. And evaluate the performance of your controllers for a step input and various disturbances.

1.4 Determining the DC gain, K_{uv}, and designing open-loop control

Velocity versus applied voltage data for one possible motor servo are seen in figure 5. These data show velocity to be a nearly linear function of voltage, except near zero volts where motor friction introduces a nonlinearity. The DC gain of the system for variations in velocity about an operating point of 12.5 [rad/sec] can be determined from the slope of the input-output function at the operating point. The slope is seen to be 7.25 [rad/sec/volt] in the case of figure 5.

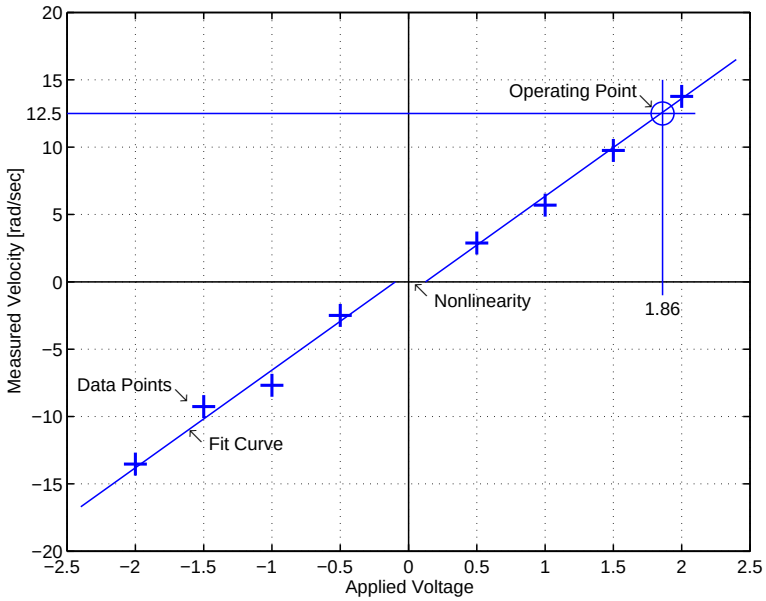


Figure 5: A plot of measured velocity as a function of applied voltage. The points '+' mark the data points, the curves are lines fit through the data points.

1.5 Open-loop controller design

To design open-loop control, start with the modeling equations

$$y(t) \simeq y_0 + K_{uv}\,\delta u(t) \tag{10}$$

$$\delta u(t) = u(t) - u_0 \tag{11}$$

and add the open-loop control law

$$u(t) = b_0 + b_1\, r(t) \tag{12}$$

Using the goal, $y(t) = r(t)$, we solve for controller gains b_o and b_1. Starting by plugging Eqns (11) and (12) into (10), one finds:

$$y(t) = y_0 + K_{uv}\,(b_0 + b_1\, r(t) - u_0) \tag{13}$$

Setting $y(t) = r(t)$ and solving for b_0 and b_1 gives:

$$b_1 = \frac{1}{K_{uv}} \tag{14}$$

$$b_0 = u_0 - \frac{1}{K_{uv}}\, y_0 \tag{15}$$

When an operating point (u_0, y_0) is specified, and the linearization K_{uv} is determined, Eqns (14) and (15) are the design equations for open loop control (our first controller !). Note that the design equations were derived by combining the linearized model and control law, and solving for $y(t) = r(t)$ (this is a common theme in controller design).

1.6 Designing P-type closed-loop control

In closed-loop control, the control law computes the control signal based on an ***error signal***. Writing in the most general form, we may write the control law as

$$u(t) = g_c\left(e(t)\right) \tag{16}$$

where $g_c(\cdot)$ is a function. For what is known as proportional-type or ***P-type*** control (used in many simple applications) $g_c(\cdot)$ is given as

$$u(t) = K_p\, e(t) \tag{17}$$

where K_p is the controller proportional gain constant.

The ***loop gain*** of a servo system is the product of the gains of all of the blocks in the feedback loop. Considering figure 4, the loop gain for the closed-loop system is:

$$G_{loop}(s) = G_c(s)\, G_{uv}(s)\ . \tag{18}$$

Plugging in $G_c(s) = K_p$ for the P-type controller, $G_{uv}(s) = K_{uv}$ for the basic plant model, and introducing $K_{loop} = G_{loop}(s = j0)$ as the DC value of the loop gain, we find

$$K_{loop} = K_p\, K_{uv}\ . \tag{19}$$

Turning around equation (19) gives an equation for designing P-type closed-loop control:

$$K_p = \frac{K_{loop}}{K_{uv}} \tag{20}$$

Note that K_{uv} is a measured quantity, and, as we will learn in lecture, K_{loop} may be set by a system accuracy requirement.

1.7 Designing P-type closed-loop control with feed forward

Equation (12) is an example of a ***feed-forward*** control term; it is a calculation of the control effort based on the reference signal $r(t)$. Equation (17) is an example of a ***feedback*** control term; it is a calculation of control effort based on $r(t)$ and the measured output of the system $y_s(t)$.

Feedback control is widely used in practice, because of its ability to correct for errors. In some controllers a feed-forward term is added to the control law, to reduce the errors that require correction. The control law for P-type closed-loop control with feed forward may be written:

$$u(t) = b_0 + b_1 r(t) + K_p e(t) \tag{21}$$

where controller parameters b_0, b_1 and K_p are given by the equations derived above.

1.8 Prelab questions

- Answer these questions as the prelab write-up.

Question 1. (True / False) The complexity of a plant model is determined by the complexity of the plant, for a given piece of equipment, there is only one correct plant model.

Question 2. What are the units of b_0 and b_1 in Eqn (12) ? Show that the units in Eqn (12) balance.

Question 3. An input-output curve for a pump is seen in figure 6. The pump is part of chiller for an industrial process with feedback control, pumping cold water through a jacket on a process vessel.

3.a (Essay Question) Could a pump element with an input-output graph like figure 6 be a useful part of a linear feedback control loop, or is the non-linearity so strong that a pump like this one will mess up any linear feedback control loop ? Describe in 1-2 paragraphs why or why not.

3.b Define K_{uf} to be the gain of the pump element, from input (volts) to flow (cfs). If the operating point corresponds to 3.0 cfs of flow, what are the values of u_0 , y_0 and K_{uf} ?

Note: The input-output slope can be estimated by sketching a tangent line at the operating point and measuring the rise over run of the tangent line. See figure 2.

3.c For the operating point of question 3.b, what is the approximate range over which the linear model prediction of $\delta y(t)$, given by $\delta y(t) = K_{uf}\, \delta u(t)$, is accurate within 10% of the true $\delta y(t)$ read from the figure.

3.d For an operating point of 0.5 cfs, what are u_0 , y_0 and K_{uf} ? What is the approximate range over which the linear model prediction is accurate within 10% of the true $\delta y(t)$?

Question 4. Starting with modeling equations (10) and (11), and control law (12), show mathematically that design equations (14) and (15) would give perfect tracking (that is, $y(t) = r(t)$) if the model is perfect and there are no disturbances.

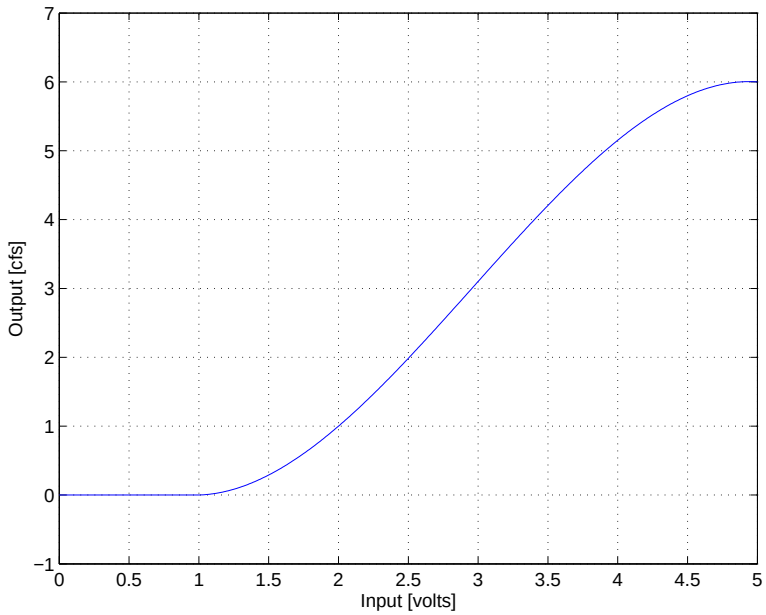


Figure 6: Input-Output curve for a pump, from volts input to cubic feet per second (cfs) of flow.

Question 5. Determine the open-loop gains, b_0 and b_1, for the system associated with the data of figure 5 using an operating point of y_0 =12.5 [rad/sec].

Question 6. Determine the applied voltage when $r(t)$ = 10.0 [rad/sec].

Question 7. K_{loop} has units of the product of blocks around the loop,

$$\text{Units } K_{loop} = \text{Units } \frac{y_s(t)}{e(t)} \tag{22}$$

What are the units of K_{loop} ?

Question 8. For the system of figure 4 (only feedback control, Eqn (17)) and data given in figure 5, determine K_p to give a loop gain of $K_{loop} = 4.0$.

Determine the applied voltage when $r(t)$ = 10.0 [rad/sec] and $y(t)$ = 9.8 [rad/sec].

Question 9. Draw the block diagram for an SRV02 system with control law of Eqn (21). (Note: your block diagram will be very close to the Simulink model you will build to test P-type control with feed-forward. This is a basic selling point of Simulink).

Question 10. Using Eqn (21) and the controller parameters determined above (K_{loop} = 4.0), calculate the applied motor voltage when $r(t)$ = 10.0 and $y(t)$ = 9.8 [rad/sec].

2 In Lab

2.1 Basic setup and sensors

2.1.1 Cabling the SRV02

Cable the SRV02 as you did in lab part 2, including the 2.5 Ω resistor block. This setup will be the standard configuration throughout the semester.

2.1.2 Setting up sensors

Copy `EE474_Lab02_Model01.mdl` to a new model, such as `Lab03_FBwFF.mdl` (or a name you choose), and setup the sensors to provide signals in physical units, as seen in figure 7. Going forward, all control experiments and plots will be in physical quantities.

Do not embed constants (numbers) in the gain and offset blocks for your sensors setup (or, indeed, anywhere in your simulation diagram). Rather, type variable names into the Simulink blocks (as seen in figure 8, below) and set the variables from a Matlab script. Write a Matlab script `SetupSRV02_Unit0x.m`, (`x` is your unit number) to setup the parameters, K_{tach}, etc., for the sensors and to compute the open- and closed-loop controller gains. Run your script whenever the parameters need to be set. See appendix 4.1.

2.2 System identification, determine the steady-state model of the SRV02

Step 1. Running the SRV02 at a steady voltage and velocity, as you did for sensor calibration, measure the steady state velocity as a function of applied voltage for a range of applied voltages.

- Collect data at a sufficient number of voltage levels to accurately estimate the slope of the voltage/velocity curve. Collect data on velocities at least down to -20 [rad/sec] and at least up to 20 [rad/sec]. (The example in figure 5 shows four points on each side of 0 volts.)
- For each collection, run the SRV02 for a sufficient interval, save the data to the Matlab workspace, and analyze the data with a routine such as `DisplayStatistics.m`.

Suggestion: Throughout the labs, save your data to .mat files (appropriately named) so that you can analyze data outside lab, reanalyze the data if needed, and make plots for your lab report.

Step 2. In Matlab, fit your voltage/velocity data to a straight line (see Appendix 4.2)

- Because the friction may be direction dependent, separately fit the positive and negative rotation direction data
- Each fit will give a slope. The slope values should be quite comparable, average the two slopes to determine K_{uv} [radians/second/volt] for your SRV02 unit.

  Note: A graph of your data from steps 1 and 2 is a Principal Graph.

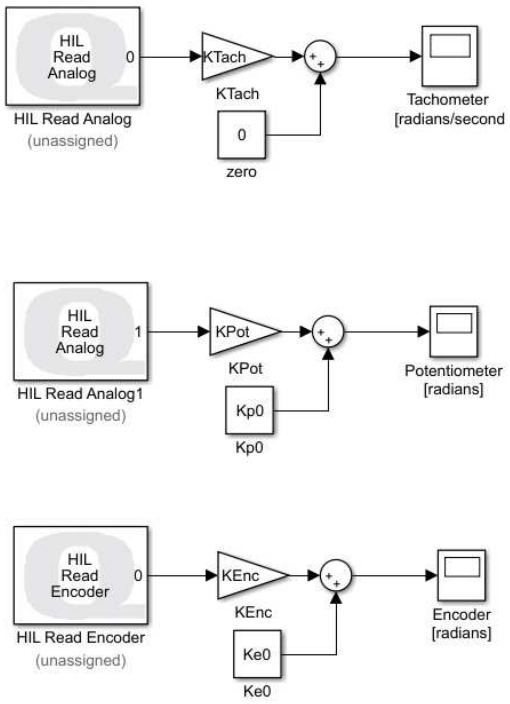


Figure 7: Sensors, setup with calibration coefficients.

2.3 Open-Loop Control

Step 3. Using your measured data from step 1 to

- Calculate the value for u_0, using $y_0 = 12.5$ [rad/sec], and
- Design the open-loop controller (determine b_0 and b_1), using the open-loop design equations, Eqns (14) and (15).

Step 4. Evaluate the performance of your open-loop controller

1. Setup a new simulink model (starting from a copy of `Lab03_FBwFF.mdl` to pickup all the details in that model). Add
   (a) A pulse generator to generate the reference command $r(t)$ as pulses from 10.0 to 15.0 [radians/second], with a period of 6 seconds and 50% duty cycle. This can be done with a pulse generator block, added to a constant signal, as seen in figure 8.
      - Put a scope on signal $r(t)$, to validate that your model generates the correct square-wave waveform. (If you copy an existing scope and change the name and the logging file name, it will have the other parameters set. The log file for this scope will be referred to as (and can be named) `rtOutput`)

   (b) Your controller, as $u = b_0 + b_1 r(t)$. This can be done with a constant block, gain block and summing junction, as also seen in figure 8.

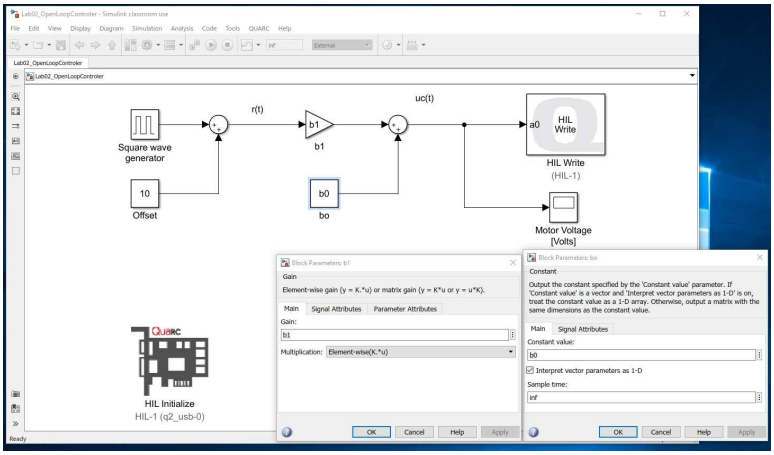


Figure 8: Open-loop controller detail (the complete simulation model will include the controller plus the sensors). Setup b_0 and b_1 as parameters, set from a Matlab script.

2. Run the model and SRV02 servo for several cycles of the square wave. The data will be logged to the Matlab workspace by the scopes.

3. Measure the average and RMS steady-state error at 10.0 [rad/sec]; and at 15.0 [rad/sec].

   - When you look at the data with `DisplayStatistics()` you will see a transient (changing value of $e(t)$, with exponential convergence) followed by steady-state. Measure the average and RMS errors in steady state.
   - A method to show average and RMS error is compute the error signal from the logged signals in Matlab, and call `DisplayStatistics()` on the signal of the $e(t)$ scope.

2.4 Closed-loop, P-type control

Step 5. Using the SRV02 gain, K_{uv}, and the p-type control design equation, design the closed-loop p-type controller (determine K_p) for a DC loop gain of $K_{loop} = 4.0$.

Step 6. Evaluate the performance of your closed-loop controller

1. Set up a new simulink model (the model can be started by copying the previous). Build the closed-loop P-type control. An example model is seen in figure 9. Here the controller gain is implemented as a gain block. A PID block can also be used, with K_d and K_i set to zero. Notice that K_p is a parameter (not an embedded constant).

2. Set the reference signal to be a square wave from 10.0 to 15.0 [rad/sec] with a period of 6.0 seconds.

3. Run a square wave event and log the data.

4. Measure the average and RMS steady-state error at 10.0 [rad/sec]; and at 15.0 [rad/sec].

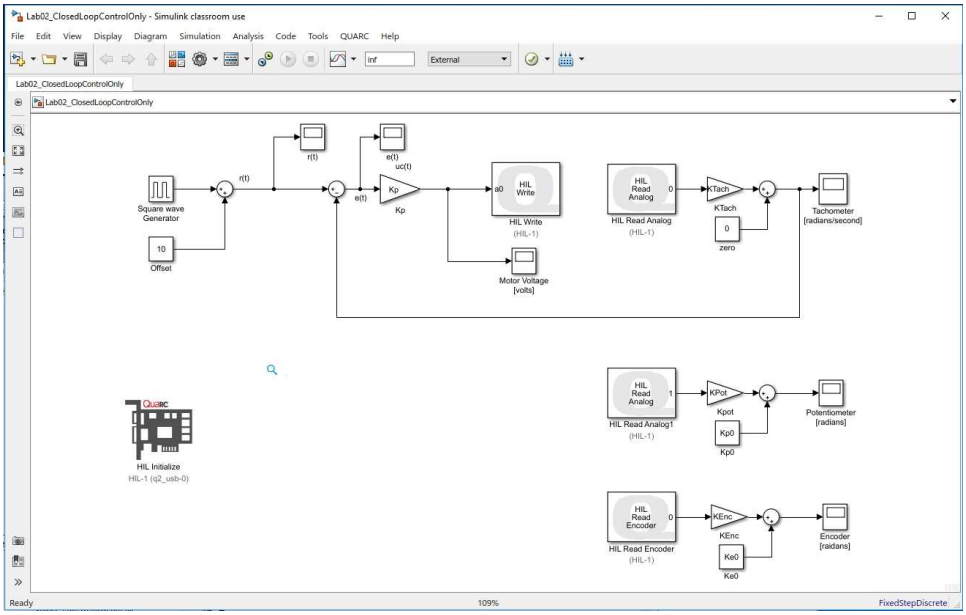


Figure 9: Closed-loop controller. Notice scopes on $r(t)$ and $e(t)$.

2.5 Closed-loop, P-type control with feed forward

Step 7. Starting with a copy of the simulink model for a previous controller, create a new controller that combines the feed-forward and feedback controllers you tested in sections 2.3 and 2.4. An example is seen in figure 10.

- Set the b_0 and b_1 terms set as for step 4, and your K_p term set as for step 6.

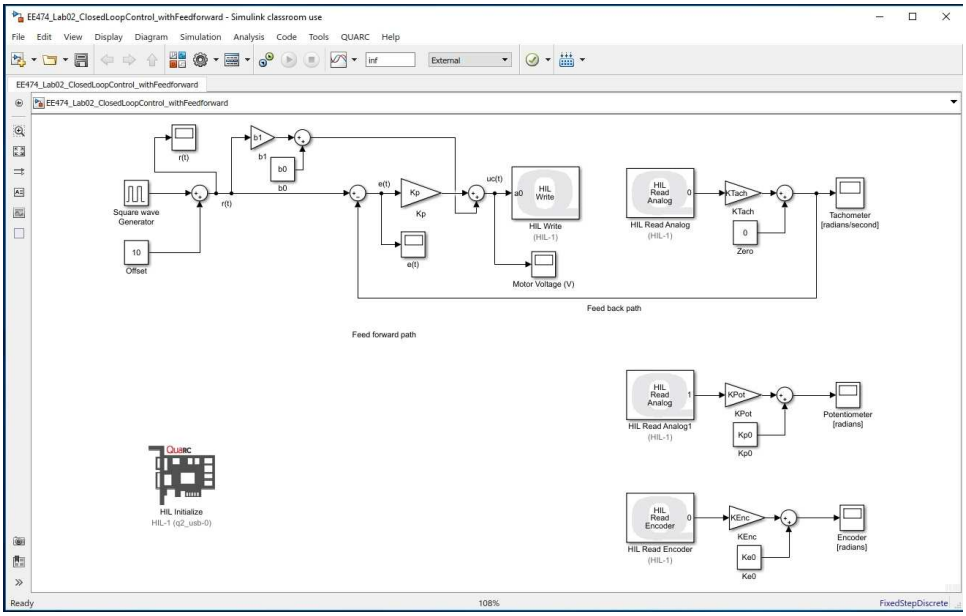


Figure 10: Closed-loop control with feed forward.

Step 8. Evaluate the performance of your closed-loop controller with feed forward.

1. Set the reference signal to be a square wave from 10.0 to 15.0 [rad/sec] with a period of 6.0 seconds.

2. Run a square wave event and log the data.

3. Measure the average and RMS steady-state error at 10.0 [rad/sec]; and at 15.0 [rad/sec].

3 Write-up

3.1 Principal Values and Graphs

- For lab part 3 the principal values are
  - K_{uv}, the right-hand rotation DC gain of your SRV02; controller parameters b_0, b_1 and K_p
  - Average RMS steady state error values for 15 [rad/sec] for the three controllers (steps 4, 6, 8)
- The principal graphs are:
  - Your plot of velocity versus voltage, collected in steps 1 and 2
- The additional data are:
  - Full chart of steady-state errors with 3 controllers and two speeds.

3.2 Questions

Include in your write up:

1. Include for each system configuration in which measurements were taken:

   (a) A block diagram, showing the configuration of the system.

   (b) Numerical values for all parameters used.

   (c) A report of experimental data, shown by plot, where appropriate.

2. A graph of the error response in each test condition (steps 4, 6 and 8), showing 2 to 3 cycles of the motion response.

3. A table of measured steady-state error characteristics for each test condition.

4. Answers to questions below.

Question 11. Which provides the most accurate control, open-loop, closed-loop, or closed-loop with feed forward ?

(Essay Question) Why ?

Question 12. Quantities in control loops have physical units (for example, [volts] or [rad/sec] for signals $u(t)$ and $y(t)$ or [volts/rad/sec] or [rad/sec/volt] for the blocks of the controller and servo).

What are the units of each of the controller parameters you designed in the lab ?

What are the units of the loop gain ?

4 Appendices

4.1 Example script to set parameters

```
%% File:  SetupSRV02_05.m
%% Parameters for SRV02 # 5,  Feb 26, 2017
%%  NOTE:  Numbers are made up, do not use these numbers
KTach = 2.1234              %% Rad/sec / volt
KPot  = 3.1234              %% Rad / volt
Kp0   = -(-4.1234) * KPot   %% Radians
KEnc  = -(2*pi)/4096        %% Rad / count
Kuv   = 5.1234

%% Setup open-loop controller
y0 = 12.5
u0 = 1.86
b1 = 1/Kuv
b0 = u0 - (1/Kuv)*y0
%% Setup closed-loop controller
KLoop = 2.0
Kp    = KLoop / Kuv
```

4.2 Fitting parameters to data

The Mathworks note

`https://www.mathworks.com/help/matlab/data_analysis/linear-regression.html`

provides a good introduction to fitting data to a linear model. Here, only the essentials are given.

If we write

$$y(k) = a_o + a_1 x(k) \tag{23}$$

where $x(k)$ is the *independent* variable, it is the variable set by the experimenter, such as applied voltage, $y(k)$ is the *dependent* (measured) variable, and k is the index on samples, $k \in \{1, 2, \cdots, n\}$, where n is the number of data points. Then we can setup the determination of a_0 and a_1 as a linear algebra problem. Putting the data in matrix-vector form:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} \tag{24}$$

(and can be named)Eqn (24) is the matrix-vector equation

$$Y = XA \tag{25}$$

In Matlab we can solve for A with

```
>> Y  = [y1; y2; y3; ...]

   >> X0 = [x1; x2; x3; ...]
   >> X = [ones(length(X0),1), X0]
   >> A = X \ Y
```

- For example, with the x and y data

  ```
  Y  = [ 8.9; 12.1; 16.2; 21.2 ]
  X0 = [ 1.5; 2.0; 2.5; 3.0]
  ```

  the calculation gives

  ```
  A =  -3.8500          %% best fit a0
        8.2000          %% best fit a1
  ```

  which indicates that the slope is 8.2 radians/second per volt, and the y-axis intercept is -3.85 [radians/second].